Setting up a Sahara cluster

The OpenStack team has provided clear documentation on how to set up Sahara services. Do follow the steps given in the installation guide to deploy Sahara in your environment. There are several ways in which the Sahara service can be deployed. To experiment with it, Kolla would be a good choice. You can also manage a Sahara project through the Horizon dashboard (*https://docs.openstack.org/sahara/latest/install/index.html).*

ETL (extract, transform and load) or ELT (extract, load and transform) with a Sahara cluster

There are numerous ETL tools available in the market and traditional data warehouses have their own benefits and limitations—they might be in some other location, rather than in your data source. The reason I am targeting Hadoop is that it is an ideal platform to run your ETL jobs. Data in your data store can come in various forms like structured, semi-structured and unstructured data. The Hadoop ecosystem has tools to ingest data from different data sources including databases, files and other data streams, and store it in a centralised Hadoop Distributed File System (HDFS). As the volume of data grows rapidly, Hadoop clusters can be scaled and you can leverage OpenStack Sahara.

Apache Hive is the data warehouse project built on top of the Hadoop ecosystem. It is a proven tool for ETL analysis. Once the data is extracted from the data sources with tools such as Sqoop, Flume, Kafka, etc, it should be cleansed and transformed by Hive or Pig scripts, using the MapReduce technique. Later we can load the data to the table we have created during the transformation phase and analyse it. This can be generated as a report and visualised with any BI tool.

Another advantage of Hive is that it is an interactive query engine and can be accessed via Hive Query Language, which resembles SQL. So a database operative can execute a job in the Hadoop ecosystem without prior knowledge of Java and MapReduce concepts. The Hive query execution engine parses the Hive query and converts it into a sequence of MapReduce/Spark jobs for a cluster. Hive can be accessed by the JDBC/ODBC driver and Thrift clients.

Oozie is a workflow engine available in the Hadoop ecosystem. A workflow is nothing but a set of tasks that needs to be executed as a sequence in a distributed environment. Oozie helps us to convert a simple workflow into cascading multiple workflows and creates coordinated jobs. However, it is ideal for creating workflows for complex ETL jobs. It does not have modules to support all actions related to Hadoop. A large organisation might have its own workflow engine to execute its tasks. We can use any workflow engine to carry out our ETL job. OpenStack Mistral (Workflow as a Service) is the best example. Apache Oozie resembles OpenStack Mistral in some aspects. It acts as a job scheduler and can be triggered at regular intervals of time.

Let us look at a typical ETL job process with Hadoop. An application stores its data in a MySQL server. The data stored in the DB server should be analysed within the shortest time and at minimum cost.

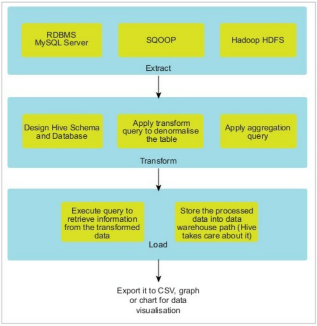

Figure 3: ETL workflow in Hadoop

The extract phase

The very first step is to extract data from MySQL and store it in HDFS. Apache Sqoop can be used to export/import from a structured data source such as an RDBMS data store. If the data to be extracted is semi-structured or unstructured, we can use Apache Flume to ingest the data from a data source such as a Web server log, a Twitter data stream or sensor data.

```
sqoop import \
        --connect jdbc:mysql://<database host>/<database
name> \
        --driver com.mysql.jdbc.Driver
        --table <table name> \
        --username <database user> \
        --password <database password> \
        --num-mappers 3 \
        --target-dir hdfs://Hadoop_sahara_local/user/
rkkrishnaa/database
```

The transform phase

The data extracted from the above phase is not in a proper format. It is just raw data, and should be cleansed by applying a proper filter and data aggregation from multiple tables. This is our staging and intermediate area for storing data in HDFS.

Now, we should design a Hive schema for each table and create a database to transform the data stored in the staging area. Typically, the data is in CSV format and each record is delimited by a comma (,).